# Projeto e Análise de Algoritmos — Lista de Exercícios I

Instituto Federal de Minas Gerais, Campus Formiga · Bacharelado em Ciência da Computação · 2026/2
Professor: Mário Luiz Rodrigues Oliveira · Aluna: Luisa Caetano

> **Convenção de custo.** Salvo menção em contrário, o custo é medido em *operações primitivas* no modelo RAM, atribuindo 1 unidade a cada atribuição, comparação e incremento, conforme o guia de análise fornecido no enunciado. Escreve-se $T(x)$ para o custo de uma entrada específica $x$ e $W(n) = \max\{T(x) : |x| = n\}$ para o custo de pior caso. Declarar a unidade importa: uma testemunha obtida contando comparações não vale automaticamente para operações primitivas.

## Questão 1

> Enunciado
>
> Responda V para os itens que são verdadeiros e F para os itens que são falsos e justifique suas respostas. Respostas sem justificativas não serão consideradas.
>
> a. Se eu provar que um algoritmo leva $O(n^2)$ no pior caso, é possível que leve $O(n)$ em *algumas* entradas?
> b. Se eu provar que um algoritmo leva $O(n^2)$ no pior caso, é possível que leve $O(n)$ em *todas* as entradas?
> c. Se eu provar que um algoritmo leva $\Theta(n^2)$ no pior caso, é possível que leve $O(n)$ em *algumas* entradas?
> d. Se eu provar que um algoritmo leva $\Theta(n^2)$ no pior caso, é possível que leve $O(n)$ em *todas* as entradas?
> e. Se eu provar que um algoritmo leva $\theta(n^2)$ no pior caso, é possível existir alguma entrada suficientemente grande para a qual o algoritmo realiza pelo menos $100 \cdot n$ operações primitivas?
> f. Se eu provar que um algoritmo leva $O(n^2)$ no pior caso, há obrigatoriamente alguma entrada suficientemente grande para a qual o algoritmo realiza pelo menos $100 \cdot n$ operações primitivas?

> **Lista revisada pelo professor** (arquivo de 09/09/2026; o cabeçalho da lista manteve a data original). O item (e) trocou $O(n^2)$ por $\theta(n^2)$, e o item (f) é novo. A versão anterior está arquivada em `listas-de-exercicios/versoes-anteriores/`.

Os seis itens giram em torno de uma única distinção: **O é teto e Θ é teto com piso**, e ambos falam do *máximo* $W(n)$, não de cada entrada individual. Por isso custo $O(n)$ em *algumas* entradas é possível — existe algoritmo em que isso ocorre —, e (a) e (c) são V com ou sem piso: o piso, quando existe, só obriga a entrada que realiza o máximo a ser cara; nada impede outra entrada de custar $O(n)$, e o insertion sort com a entrada já ordenada é uma testemunha (itens (a) e (c) abaixo). Já custo $O(n)$ em *todas* as entradas é incompatível com um piso de ordem $n^2$: (b) é V porque O não impõe piso nenhum, e (d) é F porque o piso $c_1 n^2$ do Θ obriga alguma entrada a custar pelo menos $c_1 n^2$, o que nenhuma cota $c \cdot n$ acompanha para $n$ grande. Em (e) e (f) mudam duas coisas de uma vez: a hipótese (θ ou O) e a pergunta ("é possível" ou "há obrigatoriamente").

### (a) V

Provar $W(n) = O(n^2)$ limita por cima o custo *máximo*. Nada é dito sobre entradas individuais, que podem custar muito menos que o teto.

Tome o *insertion sort*, contando comparações de chave. Em toda entrada de tamanho $n$ o algoritmo executa no máximo $n(n-1)/2$ comparações, logo $W(n) \le \frac{1}{2}n^2$ para todo $n \ge 1$: temos $W(n) = O(n^2)$ com $c = \frac{1}{2}$ e $n_0 = 1$. Já numa entrada previamente ordenada em ordem crescente, o teste do `while` falha na primeira avaliação de cada iteração do `for`, e o custo é exatamente $n-1$ comparações, isto é, $O(n)$ com $c = 1$ e $n_0 = 1$.

As duas coisas convivem sem contradição: $\frac{1}{2}n^2$ é teto de todas as entradas, e as entradas ordenadas ficam bem abaixo dele. **Logo, é possível.**

### (b) V

O ponto aqui é que **um teto pode ser folgado**. Provar $W(n) = O(n^2)$ não afirma que exista entrada custando da ordem de $n^2$; a notação O não impõe piso nenhum.

De fato, custar $O(n)$ em todas as entradas já *implica* $O(n^2)$ no pior caso: se $T(x) \le c \cdot n$ para toda entrada $x$ de tamanho $n \ge n_0$, então $W(n) \le c \cdot n \le c \cdot n^2$ para todo $n \ge \max(n_0, 1)$, pois $n \le n^2$ quando $n \ge 1$.

Exemplo concreto: a busca linear executa no máximo $n$ comparações em qualquer entrada, logo é $O(n)$ em *todas* elas ($c = 1$, $n_0 = 1$) — e também é $O(n^2)$ no pior caso. A prova de $O(n^2)$ simplesmente não era justa. **Logo, é possível.**

### (c) V

$W(n) = \Theta(n^2)$ significa que existem $c_1, c_2 > 0$ e $n_0$ com $c_1 n^2 \le W(n) \le c_2 n^2$ para todo $n \ge n_0$. O piso $c_1 n^2$ incide sobre o **máximo**: ele obriga a existir, para cada $n$ grande, *alguma* entrada cara. Não obriga que *toda* entrada seja cara.

Insertion sort de novo, em comparações. No pior caso (vetor estritamente decrescente) a iteração $j$ executa exatamente $j-1$ comparações *de chave*: são $j$ avaliações do teste do `while`, mas a última sai por esgotamento do índice ($i = 0$) e nem chega a comparar chaves. Somando, $\Sigma_{j=2..n}\,(j-1) = n(n-1)/2$. Para $n \ge 2$ vale $n-1 \ge n/2$, logo $W(n) \ge \frac{1}{4}n^2$; e sempre $W(n) \le \frac{1}{2}n^2$. Testemunhas: $c_1 = \frac{1}{4}$, $c_2 = \frac{1}{2}$, $n_0 = 2$.

Ainda assim, a entrada já ordenada custa $n-1$ comparações, que é $O(n)$. **Logo, é possível.**

### (d) F

Aqui o quantificador é universal, e ele colide frontalmente com o piso do Θ.

Suponha as duas hipóteses simultaneamente. De $W(n) = \Theta(n^2)$ vêm $c_1 > 0$ e $n_1$ com $W(n) \geq c_1 n^2$ para $n \geq n_1$. De "$O(n)$ em *todas* as entradas" vêm $c > 0$ e $n_2$ com $T(x) \leq c \cdot n$ para toda entrada de tamanho $n \geq n_2$; como $W(n)$ é o máximo de um conjunto de valores todos limitados por $c \cdot n$ — conjunto finito e não vazio, logo o máximo existe —, segue $W(n) \leq c \cdot n$.

$$\text{Para todo } n \geq n_0 = \max(n_1, n_2): \quad c_1 n^2 \leq W(n) \leq c \cdot n \implies c_1 n^2 \leq c \cdot n \implies n \leq c/c_1.$$

Isto é, *todo* $n \geq n_0$ teria de satisfazer $n \leq c/c_1$, o que é falso: basta tomar $n^* = \max(n_0, \lfloor c/c_1 \rfloor + 1)$, que cumpre $n^* \geq n_0$ mas $n^* > c/c_1$. Contradição.

> O passo decisivo é $W(n) \leq c \cdot n$, e ele só existe porque o limite $O(n)$ foi suposto para **todas** as entradas. Em (c), onde apenas *algumas* entradas eram $O(n)$, esse passo não pode ser dado — é exatamente aí que os dois itens se separam.

**Logo, é impossível.**

### (e) V

**Duas leituras do enunciado.** "Alguma entrada suficientemente grande para a qual realiza pelo menos 100n operações" pode querer dizer (L1) que existe *alguma* entrada $x$, de tamanho $n$ qualquer, com $T(x) \geq 100 \cdot n$; ou (L2) que, para *todo* $n$ a partir de algum ponto, existe entrada de tamanho $n$ com $T(x) \geq 100 \cdot n$. (L2) implica (L1). Aqui se prova (L2), e em (f) se refuta (L1); por isso as duas respostas valem em qualquer das leituras.

Suponha $W(n) \in \theta(n^2)$, com testemunhas $c_1, c_2 > 0$ e $n_0 \geq 1$: $c_1 n^2 \leq W(n) \leq c_2 n^2$ para todo $n \geq n_0$. Defina

$$N = \max(n_0, \lfloor 100/c_1 \rfloor + 1).$$

Para todo $n \geq N$: de $n \geq \lfloor 100/c_1 \rfloor + 1 > 100/c_1$ vem $c_1 \cdot n > 100$ e, multiplicando por $n > 0$, $c_1 \cdot n^2 > 100 \cdot n$; e de $n \geq n_0$ vem $W(n) \geq c_1 \cdot n^2$. Encadeando, $W(n) > 100 \cdot n$. Como $W(n)$ é o máximo de $T(x)$ sobre as entradas de tamanho $n$, ele é atingido por alguma entrada $x_n$, e $T(x_n) = W(n) > 100 \cdot n$. (Se se preferir definir $W(n)$ como supremo, o argumento continua: como o supremo passa de $100 \cdot n$, pela definição de supremo existe $x$ com $T(x) > 100 \cdot n$.)

Portanto, para *todo* $n \geq N$ existe uma entrada de tamanho $n$ com mais de $100 \cdot n$ operações — não só "é possível": é obrigatório. Quem força isso é o piso $c_1 n^2$ do θ. **Logo, é possível (e até obrigatório).**

> **Exemplo numérico.** Insertion sort, contando só as comparações de chave: no pior caso (vetor em ordem decrescente) são $C(n) = n(n-1)/2$ comparações, e $C(n) \in \theta(n^2)$ com $c_1 = 1/4$, $c_2 = 1/2$, $n_0 = 2$ (para $n \geq 2$, $n - 1 \geq n/2$, logo $n(n-1)/2 \geq n^2/4$; e $n(n-1)/2 \leq n^2/2$ para todo $n \geq 1$). Então $N = \max(2, \lfloor 100/(1/4) \rfloor + 1) = \max(2, 401) = 401$ e, para $n \geq 401$, $C(n) \geq n^2/4 > 100n$. Como cada comparação é pelo menos uma operação primitiva, o número de operações primitivas da entrada de pior caso também passa de $100n$. O limiar $N$ da demonstração é suficiente, não o menor: as comparações da entrada de pior caso já atingem $100n$ em $n = 201$, onde $201 \cdot 200/2 = 20\,100 = 100 \cdot 201$.
> *Fencepost:* com $N = 400$, o elo $c_1 n^2 > 100n$ da demonstração falharia, pois $(1/4) \cdot 400^2 = 40\,000 = 100 \cdot 400$ — embora neste exemplo $C(400) = 79\,800$ ainda passe de $100 \cdot 400$. É o elo genérico que exige o "+1" em $\lfloor 100/c_1 \rfloor + 1$.

### (f) F

Agora só se sabe $W(n) \in O(n^2)$: um teto, sem piso. Um teto não obriga nenhuma entrada a ser cara.

**Contraexemplo.** O algoritmo que, recebendo um vetor $A[1..n]$ com $n \geq 1$, devolve $A[1]$. Ele executa um número fixo $K$ de operações primitivas em toda entrada, independente de $n$; com a convenção de custo unitário, $K$ é pequeno (acessar $A[1]$ e devolver: $K \leq 3$). O que o argumento usa é só que $K$ é uma constante com $K < 100$, qualquer que seja a convenção. Então:

- $W(n) = K \leq K \cdot n^2$ para todo $n \geq 1$, logo $W(n) \in O(n^2)$ com $c = K$ e $n_0 = 1$ (e $W(n) \geq 0$);
- para toda entrada $x$ de tamanho $n \geq 1$: $T(x) = K < 100 \leq 100 \cdot n$.

Portanto *nenhuma* entrada, de nenhum tamanho, atinge $100 \cdot n$ operações — nem na leitura (L1). A hipótese "O(n²) no pior caso" é satisfeita e a conclusão falha; logo ela não é obrigatória. **Logo, não há obrigatoriamente.**

> O contraexemplo não depende do algoritmo ser constante: qualquer algoritmo que gaste no máximo $a \cdot n + b$ operações em toda entrada de tamanho $n \geq 1$, com constantes $a, b \geq 0$ e $a + b < 100$, também serve, pois para $n \geq 1$ vale $a \cdot n + b \leq (a + b) \cdot n < 100 \cdot n$, e $a \cdot n + b \leq (a + b) \cdot n^2$ dá $O(n^2)$. É o caso da busca linear em vetor não vazio: na contagem usual, com laço `for`, a busca sem sucesso gasta $3n + 3$ operações ($4n + 3$ se o acesso a $A[i]$ contar à parte), e $4 + 3 < 100$.

> **Por que (e) e (f) diferem.** Em (e) a hipótese é θ(n²), que tem piso $c_1 n^2$; como $n^2$ cresce mais que $100n$, o piso ultrapassa $100n$ para todo $n \geq N = \max(n_0, \lfloor 100/c_1 \rfloor + 1)$. Em (f) a hipótese é $O(n^2)$, que é só teto, e o teto é compatível com custo constante.
> **Sobre a versão original do item (e).** Na versão anterior da lista, o item (e) dizia "O(n²)" (e não θ(n²)) com "é possível existir". Aquela versão também era V, mas por outro motivo: um algoritmo que faz exatamente $100n$ operações em toda entrada é $O(n^2)$ ($100n \leq 100n^2$ para $n \geq 1$). A versão nova troca O por θ e mantém "é possível"; o item (f) mantém O e troca "é possível" por "há obrigatoriamente" — e essa troca inverte a resposta. Levar para o (f) o V do (e) antigo perde o item.

## Questão 2

> Enunciado
>
> Considere $n$ representando um valor variável e $k$ um valor **constante**. Analise os códigos abaixo e encontre a complexidade assintótica em cada caso usando a notação Big O. Dê um limite assintótico **tão firme quanto possível**. Ao realizar a análise, considere o pior caso. Justifique suas respostas. Respostas sem justificativas não serão consideradas.

Todo laço `for (x = a; x <= b; x = x+1)` com $m$ iterações custa $1$ (inicialização) + $(m+1)$ testes + $m$ incrementos + $m\cdot$(custo do corpo), ou seja $2m + 2 + m\cdot\text{corpo}$. É a regra 4 do guia do enunciado, e é dela que sai tudo abaixo. **Atenção ao ponto que mais engana:** o custo de um laço inclui os testes de condição, que ocorrem *mesmo quando o corpo não executa nenhuma vez*.

### Código 1 — O(n²), e o limite é justo: Θ(n²)

```
soma = 0;
for (i = 1; i <= n; i = i + 1)
    for (j = 1; j <= n; j = j + 1)
        soma = soma + 1;
```

O laço interno tem $n$ iterações e corpo de custo 1, logo custa $2n + 2 + n = 3n + 2$. O laço externo tem $n$ iterações e esse corpo:

$$T_1(n) = 1 + [2n + 2 + n(3n + 2)] = 3n^2 + 4n + 3.$$

Como $3n^2 + 4n + 3 \le 10n^2$ para todo $n \ge 1$, vale $T_1(n) = O(n^2)$ com $c = 10$, $n_0 = 1$; e $T_1(n) \ge 3n^2$, logo também $\Omega(n^2)$. Portanto $\Theta(n^2)$.

### Código 2 — O(n²), e o limite é justo: Θ(n²)

```
soma = 0;
for (i = 1; i <= n; i = i + 1)
    for (j = 1; j <= n; j = j + 1)
        for (c = j; c <= k; c = c + 1)
            soma = soma + 1;
```

Este é o código-armadilha da questão, e ele engana nos *dois* sentidos.

**O laço mais interno quase não contribui.** Ele vai de $c = j$ até $k$, com $k$ constante, logo executa $r_j = \max(0, k - j + 1)$ vezes. Como $j$ percorre até $n$, a partir de $j = k+1$ ele **não executa nenhuma vez**. Somando sobre $j$, para $n \ge k$:

$$K = \Sigma_{j=1..n}\, r_j = k + (k-1) + \cdots + 1 = k(k+1)/2 \ \text{— uma constante, independente de n.}$$

**Mas o laço do meio continua custando n por iteração de i.** Ele é executado $n$ vezes e cada iteração paga o teste da condição e o incremento, tenha ou não trabalho interno. Custo do laço $j$: $1 + (n+1) + n + \Sigma_j(3r_j + 2) = 4n + 2 + 3K$. Logo

$$T_2(n) = 1 + [2n + 2 + n(4n + 2 + 3K)] = 4n^2 + (4 + 3K)n + 3, \quad \text{com } K = k(k+1)/2, \text{ válido para } n \ge k.$$

Como $K$ é constante, $T_2(n) = \Theta(n^2)$: o teto sai de $T_2(n) \le (11 + 3K)n^2$ e o piso de $T_2(n) \ge 4n^2$, ambos para $n \ge \max(1, k)$. A constante 11 não é folga gratuita: com $c = 7 + 3K$ a desigualdade falharia em $n = 1$ sempre que $k \le 1$, pois $T_2(1) = 11 + 3K > 7 + 3K$.

> Duas conclusões erradas são comuns aqui. Dizer $O(n^3)$ por haver três laços aninhados é frouxo — o enunciado pede o limite *mais firme possível*, e o terceiro laço só contribui uma constante. E dizer $\Theta(n)$, contando apenas as execuções de `soma = soma + 1`, ignora os $n^2$ testes do laço $j$, que dominam o custo.

### Código 3 — O(n²), e o limite é justo: Θ(n²)

```
soma = 0;
for (i = 1; i <= n; i = i + 1)
    for (j = 1; j <= i; j = j + 1)
        soma = soma + 1;
```

O laço interno agora tem $i$ iterações, custando $2i + 2 + i = 3i + 2$. Somando sobre o laço externo e usando $\Sigma\, i = n(n+1)/2$:

$$T_3(n) = 1 + [2n + 2 + \Sigma_{i=1..n}(3i + 2)] = 1 + 2n + 2 + 3\cdot n(n+1)/2 + 2n = (3/2)n^2 + (11/2)n + 3.$$

Teto: $T_3(n) \le 10n^2$ para $n \ge 1$. Piso: $T_3(n) \ge (3/2)n^2$. Portanto $\Theta(n^2)$.

> O laço interno é executado em média $(n+1)/2$ vezes, e não $n$ — o somatório $\Sigma\, i = n(n+1)/2$ é a origem da constante $3/2$ em vez de $3$. A *classe* é a mesma do código 1; o que muda é o fator constante.

### Código 4 — $O(n^2)$, e o limite é justo: $\Theta(n^2)$

```
soma = 0;
for (i = 1; i <= n; i = i + 1)
    for (j = i; j <= n; j = j + 1) {
        c := 1;
        while (c <= k) { soma = soma + 1;  c = c * 2; }
    }
```

**O `while` é O(1).** Ele duplica $c$ a partir de 1 até ultrapassar $k$, logo executa $m_k = \lfloor lg\ k \rfloor + 1$ vezes, **supondo k ≥ 1** (para $k \le 0$ o teste falha de saída e o laço não executa, isto é, $m_k = 0$; note que $lg\ k$ nem está definido aí). Como $k$ é *constante*, $m_k$ é constante: o bloco formado pela atribuição `c := 1` seguida do `while` custa $3m_k + 2 = O(1)$ — o `while` sozinho custa $3m_k + 1$, e a unidade restante é a atribuição, que fica fora dele.

O laço $j$ vai de $i$ até $n$, portanto tem $L = n - i + 1$ iterações, e $\Sigma_{i=1..n}\ L = n(n+1)/2$. Fazendo a conta:

$$T_4(n) = (3m_k + 4)\cdot n(n+1)/2 + 4n + 3.$$

O par de laços forma um triângulo: são $n(n+1)/2$ pares $(i, j)$, cada um com custo constante.

*Testemunhas.* De $n(n+1)/2 \ge n^2/2$ vem o piso $T_4(n) \ge ((3m_k+4)/2)\cdot n^2$, com $c_1 = (3m_k+4)/2$ e $n_0 = 1$. De $n(n+1)/2 \le n^2$, $4n \le 4n^2$ e $3 \le 3n^2$ (todas para $n \ge 1$) vem o teto $T_4(n) \le (3m_k+11)\cdot n^2$, com $c_2 = 3m_k+11$ e $n_0 = 1$ — teto justo, com igualdade em $n = 1$. Portanto $\Theta(n^2)$, o mesmo do código 3, apenas com constante maior.

### Código 5 — $O(n^2 \log n)$, e o limite é justo: $\Theta(n^2 \lg n)$

```
soma = 0;
for (i = 1; i <= n; i = i + 1)
    for (j = 1; j <= n; j = j + 1) {
        c = 1;
        while (c <= n) { soma = soma + 1;  c = c * 5; }
    }
```

**Aqui está a diferença em relação ao código 4:** o `while` testa contra $n$, não contra $k$. Multiplicando por 5 a cada passo, ele executa $m(n) = \lfloor \log_5 n \rfloor + 1$ vezes, que *depende de n* e vale $\Theta(\log n)$. Note que a base é irrelevante assintoticamente, pois $\log_5 n = lg\ n\ /\ lg\ 5$.

Os dois laços externos dão $n^2$ pares $(i, j)$, cada um pagando o `while`:

$$T_5(n) = n^2\cdot(3m(n) + 4) + 4n + 3 = \Theta(n^2 \log n).$$

Formalmente, de $\log_5 n \le m(n) \le \log_5 n + 1$ saem o piso $T_5(n) \ge (3/lg\ 5)\cdot n^2\ lg\ n$ e o teto $T_5(n) \le 12n^2\ lg\ n$ para $n \ge 6$.

| Código | Custo exato (operações primitivas) | Resposta (Big O, mais firme) | Por quê |
|---|---|---|---|
| 1 | $3n^2 + 4n + 3$ | **$O(n^2)$** | dois laços completos, corpo O(1) |
| 2 | $4n^2 + (4+3K)n + 3$ | **$O(n^2)$** | 3º laço só soma constante K = k(k+1)/2; os $n^2$ testes do laço j dominam |
| 3 | $(3/2)n^2 + (11/2)n + 3$ | **$O(n^2)$** | triângulo: $\Sigma i = n(n+1)/2$ |
| 4 | $(3m_k+4)\cdot n(n+1)/2 + 4n + 3$ | **$O(n^2)$** | triângulo × while constante (k é constante) |
| 5 | $n^2(3m(n)+4) + 4n + 3$ | **$O(n^2 \log n)$** | while depende de n: $\Theta(\log n)$ por par (i, j) |

## Questão 3

> Enunciado
>
> Demonstre as seguintes propriedades relativas à notação assintótica.
>
> a. $f(n) = 2117$, $f(n) \in O(1)$
> b. $f(n) = n^2 + 2n$, $f(n) \in O(n^2)$
> c. $f(n) = n^2 - 2n + 3$, $f(n) \in \Theta(n^2)$
> d. $f(n) = 5n$, $f(n) \in \Omega(n)$
> e. $f(n) = n^2 + 2n - 8$, $f(n) \in \Omega(n^2)$

As três definições do CLRS, que é o que se deve exibir em cada item:

| Notação | Definição | O que exibir |
|---|---|---|
| $O(g(n))$ | $\exists\ c > 0, \exists\ n_0 > 0 : 0 \le f(n) \le c\cdot g(n), \forall\ n \ge n_0$ | um c e um $n_0$ |
| $\Omega(g(n))$ | $\exists\ c > 0, \exists\ n_0 > 0 : 0 \le c\cdot g(n) \le f(n), \forall\ n \ge n_0$ | um c e um $n_0$ |
| $\Theta(g(n))$ | $\exists\ c_1, c_2 > 0, \exists\ n_0 > 0 : 0 \le c_1\cdot g(n) \le f(n) \le c_2\cdot g(n), \forall\ n \ge n_0$ | $c_1$, $c_2$ e $n_0$ |

Duas exigências que derrubam prova: as constantes são escolhidas *antes* e **não podem depender de n**; e a definição exige $f(n) \geq 0$ na região $n \geq n_0$ — o que restringe a escolha de $n_0$ nos itens com termo negativo.

**(a) $f(n) = 2117 \in O(1)$**

**Testemunhas: $c = 2117$ e $n_0 = 1$.**

Para todo $n \geq 1$ tem-se $0 \leq 2117 \leq 2117 \cdot 1$, com igualdade. A desigualdade não depende de $n$ porque $f$ é constante, então vale trivialmente em toda a região. Qualquer $c \geq 2117$ também serve. ∎

**(b) $f(n) = n^2 + 2n \in O(n^2)$**

**Testemunhas: $c = 3$ e $n_0 = 1$.**

Para $n \geq 1$ vale $n \leq n^2$, logo $2n \leq 2n^2$. Somando $n^2$ aos dois lados:

$$0 \leq n^2 + 2n \leq n^2 + 2n^2 = 3n^2, \quad \text{para todo } n \geq 1.$$

A não-negatividade é imediata, pois $n^2 + 2n > 0$ para $n \geq 1$. ∎

**(c) $f(n) = n^2 - 2n + 3 \in \Theta(n^2)$**

**Testemunhas: $c_1 = 1/2$, $c_2 = 2$ e $n_0 = 1$.**

*Não-negatividade.* O discriminante de $n^2 - 2n + 3$ é $4 - 12 = -8 < 0$ e o coeficiente líder é positivo, logo $f(n) > 0$ para todo $n$ real. Nenhuma restrição vem daqui.

*Limite superior* ($c_2 = 2$). Basta $n^2 - 2n + 3 \leq 2n^2$, isto é, $0 \leq n^2 + 2n - 3$, ou $0 \leq (n+3)(n-1)$, verdadeiro para todo $n \geq 1$.

*Limite inferior* ($c_1 = 1/2$). Basta $\frac{1}{2}n^2 \leq n^2 - 2n + 3$, isto é, $0 \leq \frac{1}{2}n^2 - 2n + 3$, ou $0 \leq n^2 - 4n + 6$. Esse polinômio tem discriminante $16 - 24 = -8 < 0$ e líder positivo, logo é positivo para todo $n$.

Assim, $\frac{1}{2}n^2 \leq n^2 - 2n + 3 \leq 2n^2$ para todo $n \geq 1$. ∎

> Observe que $c_2 = 1$ *não* funcionaria com $n_0 = 1$: em $n = 1$ temos $f(1) = 2 > 1 = 1 \cdot g(1)$. Com $c_2 = 1$ seria preciso $n_0 = 2$. A checagem em $n = n_0$ exatamente é o que separa uma prova correta de uma quase-correta.

**(d) $f(n) = 5n \in \Omega(n)$**

**Testemunhas: $c = 5$ e $n_0 = 1$.**

Para todo $n \geq 1$ vale $0 \leq 5 \cdot n \leq 5n$, com igualdade. Qualquer $0 < c \leq 5$ também serve, pois então $c \cdot n \leq 5n$. ∎

**(e) $f(n) = n^2 + 2n - 8 \in \Omega(n^2)$**

**Testemunhas: $c = 1$ e $n_0 = 4$.**

É preciso $0 \leq 1 \cdot n^2 \leq n^2 + 2n - 8$. A segunda desigualdade equivale a $0 \leq 2n - 8$, isto é, $n \geq 4$. A primeira exige $f(n) \geq 0$: como $f(n) = (n+4)(n-2)$, isso vale para $n \geq 2$, e em particular para $n \geq 4$. Logo $n^2 \leq n^2 + 2n - 8$ para todo $n \geq 4$. ∎

> **Por que $n_0 = 4$ e não menos.** Este item é o que mais derruba prova, porque o $n_0$ é justo — não há folga:

| $n$ | $f(n) = n^2+2n-8$ | $c \cdot g(n) = n^2$ | $n^2 \leq f(n)$? |
|---|---|---|---|
| 1 | $-5$ | 1 | não — e $f(n)$ é *negativa*, violando $0 \leq f(n)$ |
| 2 | 0 | 4 | não |
| 3 | 7 | 9 | não |
| 4 | 16 | 16 | sim, com igualdade |

> Escrever "$n_0 = 1$" aqui é erro, e é um erro que passa despercebido numa leitura rápida. Alternativamente, com $c = 1/2$ bastaria $n_0 = 2$: $\frac{1}{2}n^2 \leq n^2 + 2n - 8$ equivale a $0 \leq n^2 + 4n - 16$, válido para $n \geq 2{,}47\ldots$, ou seja, $n \geq 3$.

## Questão 4

> Enunciado
>
> Suponha que um algoritmo para um certo problema consome $O(n^2)$ unidades de tempo, sendo $n$ o tamanho de uma instância. Meu amigo diz ter outro algoritmo para o mesmo problema que consome apenas $\Omega(n)$ unidades de tempo. Devo ficar impressionado(a)?

**Não.** A afirmação do amigo é quase vazia, e o "apenas" do enunciado é justamente a isca.

**$\Omega$ é limite inferior.** Dizer que um algoritmo consome $\Omega(n)$ é dizer que ele gasta *pelo menos* da ordem de $n$. Isso não é promessa de eficiência — é o contrário: é uma garantia sobre o quão ruim ele é, no mínimo. Para se impressionar seria preciso um **teto** (uma afirmação em O) melhor que $O(n^2)$, e o amigo não deu nenhum.

**A afirmação é compatível com um algoritmo péssimo.** Um algoritmo $\Theta(n^5)$ também é $\Omega(n)$, pois $n^5 \geq n$ para $n \geq 1$. O mesmo vale para $\Theta(2^n)$. Ou seja, a frase do amigo não exclui nenhum desses casos.

**O que ela de fato informa.** Rigorosamente, ela não é *totalmente* vazia: ela exclui que o algoritmo dele seja sublinear — algoritmos como a busca binária, que é $\Theta(\lg n)$ e portanto *não* é $\Omega(n)$, ficam de fora. Só que essa é a única informação carregada, e ela é *desfavo-*

*rável* a ele: o amigo está dizendo que seu algoritmo não é sublinear.

> Cuidado com o argumento tentador "todo algoritmo que lê a entrada inteira já é $\Omega(n)$, logo a frase não diz nada". A implicação vale só nessa direção: ler a entrada inteira $\Longrightarrow \Omega(n)$. A recíproca é falsa, e existem problemas que admitem algoritmos sublineares. Por isso a formulação correta é a do parágrafo acima.

## Questão 5

> Enunciado
>
> Meu algoritmo para um certo problema consome O(n²) unidades de tempo, sendo $n$ o tamanho de uma instância. Meu amigo diz ter outro algoritmo para o mesmo problema que consome O(n·lg n) unidades de tempo. Devo ficar impressionado(a)?

**Sim, com ressalvas.** Desta vez a comparação é ao menos entre grandezas do mesmo tipo (dois tetos), e $n \lg n$ é assintoticamente melhor que $n^2$. Mas atenção: das duas afirmações em O, tomadas isoladamente, não se conclui ganho nenhum — é a Ressalva 1 abaixo, e ela é o coração da questão.

**O ganho assintótico é real.** Vale $n \lg n = o(n^2)$, e a recíproca falha: $n^2$ *não* é $O(n \lg n)$. De fato, suponha por absurdo que existam $c > 0$ e $n_0 \geq 2$ (podemos supor $n_0 \geq 2$, para que $\lg n > 0$) com $n^2 \leq c \cdot n \lg n$ para $n \geq n_0$. Dividindo por $n \lg n > 0$:

$$n / \lg n \leq c, \text{ para todo } n \geq n_0.$$

Mas $n / \lg n$ é ilimitada: tomando $n = 2^k$ temos $n / \lg n = 2^k/k$, e como $2^k \geq k^2$ para $k \geq 4$, segue $2^k/k \geq k$. Basta então escolher $k > \max(c, 4)$ com $2^k \geq n_0$ para contradizer a desigualdade. ∎

**Ressalva 1 — O é teto, dos dois lados.** Só posso *concluir* que há ganho se o meu $O(n^2)$ for justo, isto é, se meu algoritmo for de fato $\Theta(n^2)$. Se minha análise foi frouxa e ele é na verdade $\Theta(n \lg n)$, as duas garantias declaradas deixam de separar os algoritmos. Note a diferença: nesse caso eu perco a capacidade de *concluir* o ganho — não que o ganho deixe de existir. Ele pode existir ainda assim (o do amigo pode ser $\Theta(n)$), eu é que não consigo mais afirmá-lo a partir das duas frases.

**Ressalva 2 — o teto dele também pode ser folgado.** Simetricamente, $O(n \lg n)$ não impede que o algoritmo do amigo seja ainda melhor, por exemplo $\Theta(n)$. A afirmação dele é um limite superior, não uma descrição exata.

**Ressalva 3 — constantes ocultas e o tamanho real de n.** A notação assintótica descarta constantes, e elas importam na prática. Se meu algoritmo custa $2n^2$ e o dele $1000 \cdot n \lg n$, o meu é mais rápido até o cruzamento: $2n^2 \leq 1000\, n \lg n$ equivale a $n \leq 500 \lg n$, o que vale para $2 \leq n \leq 6311$. O cruzamento real é $n^* \approx 6311{,}93$, de modo que só a partir de $n = 6312$ o dele passa à frente. Se as instâncias reais forem menores que isso, o ganho assintótico não se converte em ganho prático.

---

**Sobre esta resolução.** As classes assintóticas da Questão 2 e as testemunhas $(c, n_0)$ da Questão 3 foram conferidas por medição: os cinco laços foram transcritos em Python e suas operações primitivas contadas por força bruta, e cada desigualdade da Questão 3 foi testada em 300 000 valores de n. Os scripts estão em `resolucoes/verificacao/`. As demonstrações foram submetidas a verificadores independentes em contexto isolado, que receberam apenas o enunciado e a prova final.

**Atualização de 09/09/2026.** Os itens (e) e (f) da Questão 1 foram refeitos para a versão revisada da lista. Texto-fonte verificado: `geradores/dados/lista01/q1ef-limpo.md`. Primeira rodada: 4 verificadores, 3 apontaram defeito numa nota lateral (a cota da busca linear estava errada na contagem usual); corrigida, a reverificação deu 4 de 4 sem defeito. Números conferidos em `verificacao/atualizacao-2026-09-chao-deterministico.py`. Lista de estudo, sem entrega — IFMG Campus Formiga, 2026/2.